

Evaluation Metrics for Regression Models

2023 | © LUIS FERNANDO TORRES

Table of Contents

- [Introduction](#)
- [Mean Absolute Error](#)
- [Mean Squared Error](#)
- [Root Mean Squared Error](#)
- [Median Absolute Error](#)
- [Maximum Error](#)
- [Mean Absolute Percentage Error](#)
- [Coefficient of Determination \(\$R^2\$ \)](#)
- [Conclusion](#)

Introduction

Regression models and techniques are extremely popular in Machine Learning across several industries. These models are efficient in accomplishing several tasks, such as:

- Estimate the price value of houses, cars, tech products, and others;
- Determine the optimal drug dosages based on the characteristics of a patient;
- Estimate the future demand for transportations services;

- Predict future sales based on historical data, events, and market trends.

In order to evaluate regression models, we have a broad range of score metrics to use. It is highly important to develop a profound comprehension of these metric scores and how to interpret them.

Although numerous packages offer built-in functions to compute these metrics, it is significantly important to know how to craft your own functions for computing them solely with native Python.

Let's go ahead and take a look at the most commonly-used metric scores to evaluate regression models.

```
In [1]: # Importing Libraries
# Data Handling
import pandas as pd

# Data Visualization
import plotly.express as px
import plotly.graph_objs as go
import plotly.subplots as sp
from plotly.subplots import make_subplots
import plotly.figure_factory as ff
import plotly.io as pio
from IPython.display import display
from plotly.offline import init_notebook_mode
init_notebook_mode.connected=True

# Machine Learning metrics
from sklearn.metrics import (
    mean_squared_error,
    r2_score,
    mean_absolute_error,
    median_absolute_error,
    mean_poisson_deviance,
    mean_gamma_deviance,
    max_error,
    mean_absolute_percentage_error
)
```

Mean Absolute Error

The Mean Absolute Error gives us the average value of the total absolute differences between the predicted values output by the model and the actual values in the dataset.

It is expressed in the same unit scale as the data measured, which makes it a straightforward metric to interpret.

Values closer to 0 are considered better.

Let's observe the formula for the MAE score and understand each component of it.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i| \quad (1)$$

- $\frac{1}{n}$: This suggests that the sum of the absolute differences between all the actual and predicted outputs will be divided by the total amount of data points in the test set. This operation gives us the average.
- $\sum_{i=1}^n$: This is the Sigma symbol, which tells us that we are obtaining the sum of the differences between predicted and actual values for every data point.
- Y_i : This represents the actual value of Y for the i^{th} data point.
- $\hat{Y}_i$: This represents the predicted value of Y output by the model for the i^{th} data point.
- $|\dots|$: This is the modulus symbol used in Mathematics to represent the absolute value. This ensures that all values are higher than 0 and gives us the magnitude of the errors.

Let's consider the following values for `y_true` and `y_pred`, representing the actual observed data for a random target variable and the predicted values for this same variable output by a regression model.

`y_true = [23.5 45.1 34.7 29.8 48.3 56.4 21.2 33.5 39.8 41.6 27.4 36.7 45.9 50.3`



`y_pred = [25.7 43.0 35.5 30.1 49.8 54.2 22.5 34.2 38.9 42.4 26.3 37.6 46.7 51.1`



```
In [2]: # Creating a list of true values
y_true = [23.5, 45.1, 34.7, 29.8, 48.3, 56.4, 21.2, 33.5, 39.8, 41.6,
          27.4, 36.7, 45.9, 50.3, 31.6, 28.9, 42.7, 37.8, 34.1, 29.5]

# Creating a list of predicted values
y_pred = [25.7, 43.0, 35.5, 30.1, 49.8, 54.2, 22.5, 34.2, 38.9, 42.4,
          26.3, 37.6, 46.7, 51.1, 33.5, 27.7, 43.2, 36.9, 33.4, 31.0]
```

We can see below a scatter plot consisting of data points across the x-axis and y-axis. In the x-axis, we have the true values, while the predicted values are shown in the y-axis.

By hovering your mouse over the points below, you will be able to see both the actual and the predicted values.

```
In [3]: plot_df = pd.DataFrame({'Actual': y_true, 'Predicted': y_pred}) # Creating dataframe with column

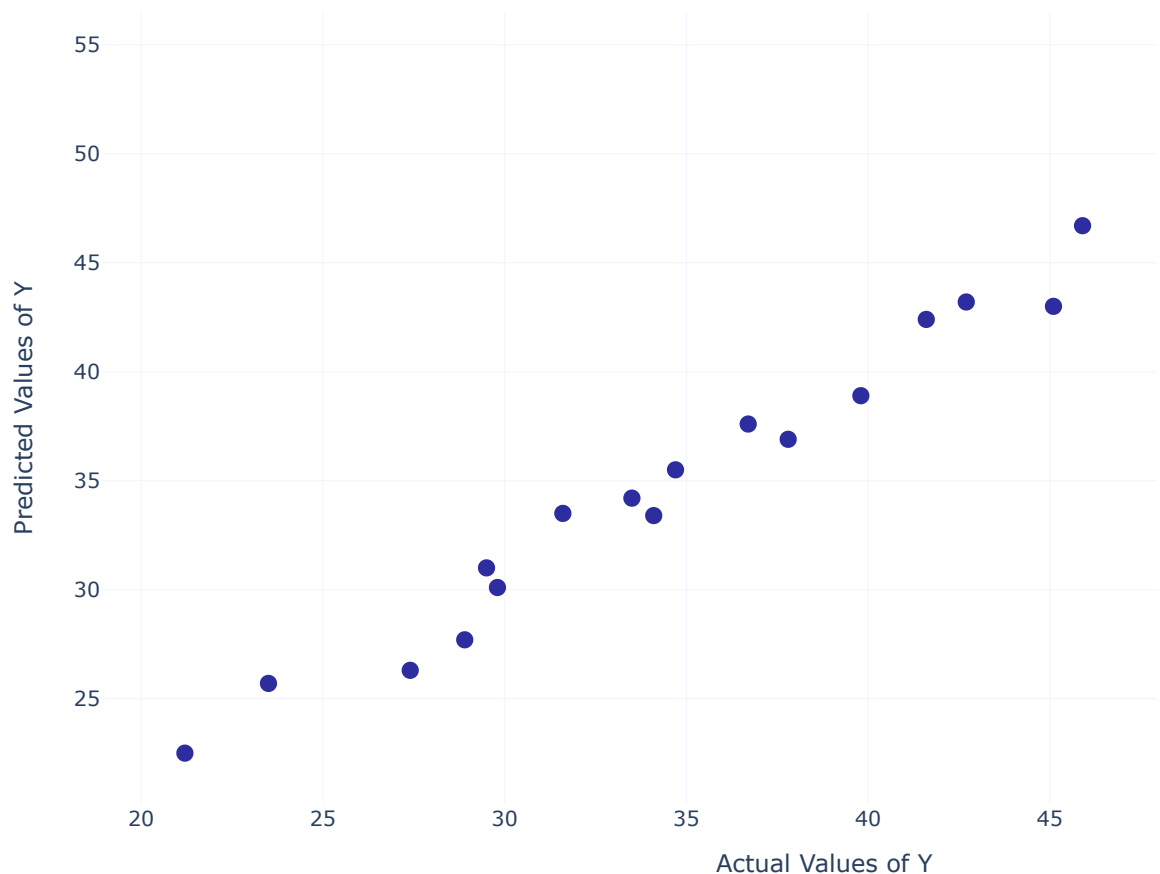
# Creating a scatter plot with Plotly
fig = px.scatter(plot_df, x='Actual', y='Predicted', opacity=0.825)
fig.update_traces(marker={'size': 10,
                          'color': 'darkblue'})

# Configuring layout
fig.update_layout(title={'text': f'<b>Actual x Predicted Values</b>',
                        'x': .025, 'xanchor': 'left', 'y': 0.968},
                  showlegend=True,
                  template = 'plotly_white',
                  height=600, width=1000)

# Renaming both the X and y axes
fig.update_xaxes(title = 'Actual Values of Y')
fig.update_yaxes(title = 'Predicted Values of Y')

# Displaying scatter plot
fig.show()
```

Actual x Predicted Values



Let's define a custom function below to compute the Mean Absolute Errors.

```
In [4]: def custom_mae(y_true, y_pred):
        absolute_sum = 0 # Initiating a variable for the accumulation of the absolute differences

        # Iterating over each data point in both y_true and y_pred
        for true, predicted in zip(y_true, y_pred):
```

```

# Subtracting predicted value from the true value to obtain the difference
absolute_error = true - predicted

# Obtaining the absolute value
# If the difference is below 0,
if absolute_error < 0:
    absolute_error = -absolute_error # We make it positive by obtaining its negation{ (-

# We add the absolute error value to the current absolute sum value
absolute_sum += absolute_error

# After iterating through every data point, we divide the absolute_sum by the total number of
mae = absolute_sum / len(y_true)

return mae # Returning value

```

We can now use the function above on the `y_true` and `y_pred` lists.

```

In [5]: # Using custom function
print('\nMean Absolute Error\n')
round(custom_mae(y_true, y_pred), 3)

```

Mean Absolute Error

Out[5]: 1.155

On average, our predictions are wrong by ± 1.155 units. So if we have a data point equal to 100, we can expect the predictions output by the model for this specific data point to fluctuate from 98.845 to 101.555.

Mean Squared Error

The formula for computing the Mean Square Error is very similar to the one we use to compute the Mean Absolute Error. This time, however, we square the differences between actual and predicted values for Y .

The Mean Squared Error may be a bit less intuitive compared to the Mean Absolute Error, especially considering that it is not expressed in the same unit scale as the data observed in `y_true`.

By squaring the differences, we penalizes larger errors more than smaller errors, making it an ideal choice to evaluate models for tasks in which larger errors may lead to undesirable outcomes.

The closer to 0, the better is the model's performance.

Let's observe the formula.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (2)$$

As you can see, it is very similar to the formula for the Mean Absolute Error. The difference is in the expression $(Y_i - \hat{Y}_i)^2$, in which we square the differences. Squaring the errors will guarantee that we do not have any negative values—so the lowest score we can get is 0—and it also gives more weight to larger differences.

Let's consider the same values we have used before for `y_true` and `y_pred`.

```
In [6]: def custom_mse(y_true, y_pred):  
  
    squared_sum = 0 # Initiating a squared sum variable equal to 0  
  
    # Iterating over y_true and y_pred  
    for true, predicted in zip(y_true, y_pred):  
  
        # Subtracting predicted from true and squaring the result  
        squared_error = (true - predicted) ** 2  
  
        # Adding the squared error result to the squared_sum variable  
        squared_sum += squared_error  
  
    # Obtaining the MSE by dividing the squared sum to the total number of data points in y_true  
    mse = squared_sum / len(y_true)  
  
    return mse # Returning result
```

```
In [7]: # Using custom function  
print('\nMean Squared Error\n')  
round(custom_mse(y_true, y_pred), 3)
```

Mean Squared Error

```
Out[7]: 1.642
```

Since the errors are squared, we **cannot** interpret the Mean Squared Error as saying that our predictions are off by 1.642 units. A more intuitive score can be obtained when we take the square root of this result.

Root Mean Squared Error

By using the Root Mean Squared Error we are able to bring the Mean Square Error back to the same unit scale as the observed data, which makes it more intuitive.

It still holds the same characteristics as the Mean Squared Error, penalizing larger errors more than smaller errors.

The closer to 0, the better is the model's performance.

Let's observe the formula.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2} \quad (3)$$

The only difference between this formula and the MSE formula is the $\sqrt{}$ symbol.

Let's consider the same values we have used before for `y_true` and `y_pred`.

```
In [8]: def custom_rmse(y_true, y_pred):  
  
    squared_sum = 0 # Initiating a squared sum variable equal to 0  
  
    # Iterating over y_true and y_pred  
    for true, predicted in zip(y_true, y_pred):  
  
        # Subtracting predicted from true and squaring the result  
        squared_error = (true - predicted)**2  
  
        # Adding the squared error result to the squared_sum variable  
        squared_sum += squared_error  
  
    # Obtaining the MSE by dividing the squared sum to the total number of data points in y_true  
    mse = squared_sum / len(y_true)  
  
    # To find the square root, we raise the mse to the power of 0.5  
    rmse = mse**0.5  
  
    return rmse # Returning result
```

```
In [9]: # Using custom function  
print('\nRoot Mean Squared Error\n')  
round(custom_rmse(y_true, y_pred), 3)
```

Root Mean Squared Error

```
Out[9]: 1.282
```

The Root Mean Squared Error gives us a result that is easier to understand. We can conclude that our predictions are off by ± 1.282 units, especially when we give more weight to larger errors.

Median Absolute Error

The Median Absolute Error measures the median value of the differences between true and predicted values for Y .

It means that at least half of the errors are less than or equal than the Median Absolute Error, while the other half greater. This is a measure of the central tendency of the errors.

This metric is robust to outliers—larger errors—and it is expressed in the same unit scale as the observed data in `y_true`, making it a straightforward metric to interpret.

Lower values are preferred over higher values for better predictive accuracy.

The formula is as below:

$$\text{MedAE} = \text{median}(|Y_i - \hat{Y}_i|) \quad (4)$$

Let's define a function to compute the Median Absolute Error and try it on our numbers.

```
In [10]: def custom_medae(y_true, y_pred):  
  
    # Creating an empty List of absolute errors  
    absolute_errors = []  
  
    # Iterating through actual and predicted values for y  
    for true, predicted in zip(y_true, y_pred):  
  
        # Computing the differences(i.e., errors)  
        error = true - predicted  
        # Obtaining the absolute value  
        if error < 0: # If the difference is a negative number,  
            error = -error # We obtain the negative of the negative, which is a positive num  
  
        absolute_errors.append(error) # Adding absolute value to the list of absolute errors  
  
    # Ordering absolute_errors List in ascending order  
    sorted_absolute_errors = sorted(absolute_errors)  
    # Obtaining the total number of elements in the sorted_absolute_errors list  
    n = len(sorted_absolute_errors)  
  
    # Obtaining the middle index of the List by dividing the total length of the List by hal  
    middle = n // 2 # Floor division to return an integer  
  
    # We must check if we have an even or odd number of elements  
    if n % 2 == 0: # If we have an even number of elements,  
        # The median will be equal to the mean of the two elements in the middle of the list  
        medae = (sorted_absolute_errors[middle - 1] + sorted_absolute_errors[middle]) / 2  
    else:  
        # For an odd number of elements, the median will be equal to the value in the middle  
        medae = sorted_absolute_errors[middle]  
    return medae  
  
In [11]: # Using custom function  
print('\nMedian Absolute Error\n')  
round(custom_medae(y_true, y_pred), 3)
```

Median Absolute Error

Out[11]: 0.9

With this result, we know that half of our predictions show a deviation of up to ± 0.9 units.

Maximum Error

For the Maximum Error Score, we compute the absolute errors between actual and predicted values and capture the largest difference between them.

This is a good metric to know what is the worst-case scenario in our validation set, the worst deviation between `y_true` and `y_pred`.

Lower values are preferred over large values.

The formula for this metric score is as below:

$$\text{Maximum Error} = \max(|Y_i - \hat{Y}_i|) \quad (5)$$

```
In [12]: def custom_max_error(y_true, y_pred):  
  
    # Creating an empty list of absolute errors  
    absolute_errors = []  
  
    # Iterating through actual and predicted values for y  
    for true, predicted in zip(y_true, y_pred):  
  
        # Computing the differences(i.e., errors)  
        error = true - predicted  
        # Obtaining the absolute value  
        if error < 0: # If the difference is a negative number,  
            error = -error # We obtain the negative of the negative, which is a positive number  
  
        absolute_errors.append(error) # Adding absolute value to the list of empty errors  
  
    # Obtaining the largest error in the absolute_errors list using the max() function  
    maximum_error = max(absolute_errors)  
  
    return maximum_error
```

```
In [13]: # Using custom function  
print('\nMaximum Error\n')  
round(custom_max_error(y_true, y_pred), 3)
```

Maximum Error

```
Out[13]: 2.2
```

In our worst-case scenario, the largest deviation between the model's output and the actual value was of ± 2.2 units. This is also a very straightforward metric, expressed in the same unit value as the observed data.

Mean Absolute Percentage Error

The Mean Absolute Percentage Error can be used for time-series data, such as forecasting sales or predicting the price of financial assets. It is expressed in percentage, making it easier to understand its results.

Let's take a look at the formula.

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|Y_i - \hat{Y}_i|}{|Y_i|} \quad (6)$$

Due to the fact that we are dividing the absolute error by the absolute value of the actual value of Y in the i^{th} data point ($|Y_i|$), we avoid using this metric when we have values that are equal to or close to 0.

```
In [14]: def custom_mape(y_true, y_pred):  
  
    # Initiating an empty variable for the sum of absolute errors  
    sum_absolute_errors = 0  
  
    # Iterating over true and predicted values  
    for actual, predicted in zip(y_true, y_pred):  
        # Computing the differences between them  
        absolute_error = actual - predicted  
  
        # Obtaining the absolute value  
        # If any number is below 0, we obtain the negative of this number to make it positive  
        if absolute_error < 0:  
            absolute_error = -absolute_error  
        # We do the same for the value in y_true  
        absolute_actual = actual  
        if absolute_actual < 0:  
            absolute_actual = -absolute_actual  
  
        # We divide the absolute error by the absolute value of y_true  
        absolute_error = absolute_error / absolute_actual  
  
        # We sum the values in absolute_error  
        sum_absolute_errors += absolute_error  
  
    # We divide the sum of absolute errors by the length of y_true to compute the MAPE score  
    mape = (sum_absolute_errors/len(y_true))  
  
    return mape
```

```
In [15]: # Using custom function  
print('\nMean Absolute Percentage Error\n')  
round(custom_mape(y_true, y_pred), 3)
```

Mean Absolute Percentage Error

Out[15]: 0.034

This result tells us that predictions deviate from the actual values by an average of 38%.

Coefficient of Determination (R^2)

The Coefficient of Determination—also referred to as R-Squared—is a measure that tells us how well a regression model fits the actual data. It quantifies the degree to which the variance in the dependent variable is predicatable from the independent variables.

Values closer to 1.0 indicate a better model.

Let's take a look at its formula and see how we can interpret it.

$$R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y}_i)^2} \quad (7)$$

- $\sum_{i=1}^n (Y_i - \hat{Y}_i)^2$: The sum of the square differences between actual and predicted values for each data point. This is also referred to as Sum of Squared Residuals
- $\sum_{i=1}^n (Y_i - \bar{Y}_i)^2$: The sum of the squared differences between actual observed value for each data point and the mean of all observed values. This captures the variance in the actual data. This is also referred to as Total Sum of Squares
- 1: This is the constant from which the fraction is subtracted to obtain the value for the R^2 between 0 and 1.

```
In [16]: def custom_rsquared(y_true, y_pred):  
  
    # Obtaining the mean of actual values  
    mean_ytrue = sum(y_true) / len(y_true)  
  
    # Obtaining the sum of the squared differences between actual and predicted valyes  
    sum_of_squared_residuals = 0  
    for true, predicted in zip(y_true, y_pred):  
        sum_of_squared_residuals += (true - predicted) ** 2  
  
    # Obtaining the total sum of squares  
    total_sum_of_squares = 0  
    for true in y_true:
```

```
total_sum_of_squares += (true - mean_ytrue) ** 2

# Computing the R-Squared score
r_squared_score = 1 - (sum_of_squared_residuals / total_sum_of_squares)

return r_squared_score
```

```
In [17]: # Using custom function
print('\nCoefficient of Determination (R²)\n')
round(custom_rsquared(y_true, y_pred), 3)
```

Coefficient of Determination (R²)

Out[17]: 0.98

A R^2 score of 0.98 indicates a high level of correlation between `y_true` and `y_pred` and it suggests that our model could explain about 98% of the variance in the observed data.

Conclusion

In conclusion, the metric scores studied above are among the most commonly-used metrics to evaluate the performance of regression models.

By getting to the end of this notebook, you might have gained insight into the Math behind these metrics and learned how to craft your own functions in Python for their computation. The idea behind this project originated from a challenge I faced in a course, where I had to manually compute metrics without using packages like Scikit-learn. This experience inspired me to create this notebook, in hopes to aid others in building a more profound understanding and ability to develop their own functions for metric computations.

It is also important to note that this notebook contains only the most popular metric scores, but there are still many more out there for you to study, such as the [weighted mean absolute percentage error](#), [Adjusted R²](#), and many others.

If you liked this notebook and feel like its content is relevant, feel free to leave your upvote. I'm also open to hear your suggestions and feedback.

Stay curious!

Luis Fernando Torres, 2023

Let's connect! 

[LinkedIn](#) • [Medium](#) • [Hugging Face](#)

Like my content? Feel free to [Buy Me a Coffee](#) 

<https://luisotorres.github.io/>